Rapport technique

Analyse de sentiment des commentaires YouTube — Du développement au déploiement cloud

TP3 — Cloud Computing · Filière INDIA · ENSAM Rabat — Université Mohammed V Année universitaire 2026/27 · Encadrante : Prof. Maria Zemzami Réalisé par : **BAMOHAMED Douaa** · 4 novembre 2026

Sommaire

1. Analyse du problème et solution · **2.** Architecture détaillée · **3.** Pipeline de données · **4.** Modélisation et choix techniques · **5.** API REST et containerisation · **6.** Extension Chrome · **7.** Résultats et métriques de performance · **8.** Tests et validation · **9.** Difficultés rencontrées et solutions · **10.** Améliorations futures · **11.** Conclusion

1. Analyse du problème et solution proposée

1.1 Contexte

Une vidéo YouTube à forte audience accumule des centaines à des milliers de commentaires. Pour un créateur de contenu, cette masse de retours constitue théoriquement une mine d'information : qu'est-ce qui a plu, qu'est-ce qui a déçu, quel format relancer. En pratique, elle est inexploitable manuellement.

Trois limites se cumulent :

- **Volume.** Lire 2 000 commentaires représente plusieurs heures de travail, à refaire à chaque publication.
- **Subjectivité.** L'impression générale qu'on retire d'une lecture rapide est fortement biaisée par les premiers commentaires lus et par les plus virulents, qui ne sont pas représentatifs.
- **Absence de quantification.** « Les retours sont plutôt bons » ne permet aucune comparaison entre deux vidéos, ni aucun suivi dans le temps.

1.2 Solution proposée

Un système MLOps complet, qui transforme un flux de commentaires non structurés en indicateurs chiffrés, directement dans l'interface où le créateur travaille déjà — son navigateur, sur la page de la vidéo.

Le système se décompose en quatre blocs :

Composant	Technologie	Responsabilité
Frontend	Extension Chrome Manifest V3	Extraire les commentaires du DOM, afficher les résultats
Backend	FastAPI + Uvicorn	Exposer `/health` et `/predict_batch`, valider les entrées
ML Engine	scikit-learn — TF-IDF + Logistic Regression	Classifier chaque commentaire en 3 classes
Déploiement	Docker sur GitHub Codespaces	Héberger l'API de façon reproductible

1.3 Flux fonctionnel

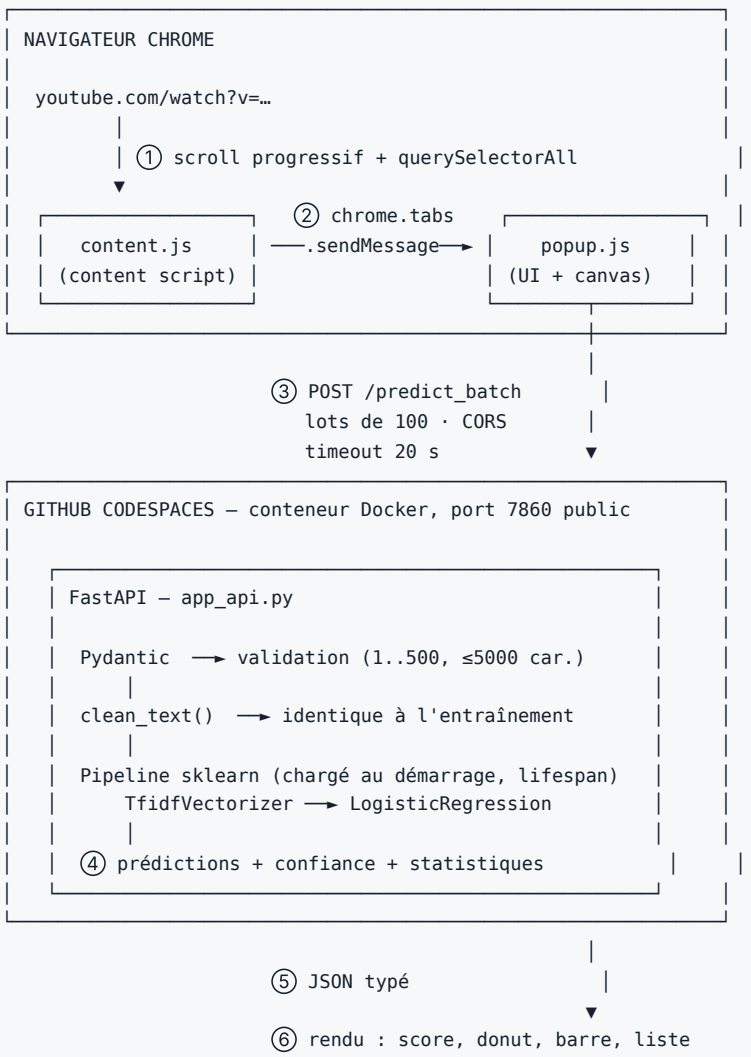
1. L'utilisateur ouvre une vidéo YouTube et clique sur l'icône de l'extension.
2. Le *content script* fait défiler la page pour déclencher le chargement paresseux des commentaires, puis lit le DOM.
3. Le popup découpe les commentaires en lots de 100 et les envoie à l'API cloud.
4. L'API nettoie chaque texte, le vectorise et prédit son sentiment.
5. L'API retourne les prédictions individuelles, leur confiance, et des statistiques agrégées sur le lot.
6. Le popup agrège les lots et affiche : score global, répartition en pourcentage, graphique en anneau, liste filtrable.

1.4 Périmètre

Classification en **trois classes** sur du texte uniquement : ni images, ni analyse d'aspect (« le son est mauvais mais le contenu est excellent » reçoit une classe unique). Le modèle est entraîné sur un corpus **Reddit**, pas YouTube — choix imposé par l'énoncé, dont le biais est documenté en section 9.

2. Architecture détaillée

2.1 Vue d'ensemble



2.2 Structure du projet

L'arborescence suit la séparation MLOps canonique **données / code / modèles** : `data/raw` (source immuable) et `data/processed` · `models/` (artefacts `.joblib`) · `src/{data,models,api,utils}` · `chrome-extension/` · `tests/` · `reports/` · `deploy/` · `docs/` · `app_api.py` et `Dockerfile` à la racine. Le détail figure dans le README.

Trois principes structurent ce découpage :

- **Une étape, un script, une sortie.** Chaque module du pipeline est exécutable indépendamment (`python -m src.data.clean_data`) et produit un artefact sur disque. On peut reprendre le pipeline à n'importe quelle étape.
- **Configuration centralisée.** `src/utils/config.py` contient tous les chemins, la graine aléatoire et les seuils. Rien n'est codé en dur ailleurs.
- **Code de production isolé.** `app_api.py` est autonome, sans dépendance à `src/`, ce qui permet une image Docker minimale.

2.3 Séparation développement / production

	Développement	Production
Point d'entrée	`src/api/main.py`	`app_api.py`
Dépendances	`requirements.txt` (13 paquets)	`requirements-prod.txt` (6)
Port	8000	7860 (standard HF)
Mode	`--reload`	serveur figé
Structure	package `src/` modulaire	fichier unique

La fonction `clean_text()` est **dupliquée** dans `app_api.py`. C'est un choix assumé : il garantit que le conteneur n'a besoin d'aucun fichier de `src/`, au prix d'une duplication que les tests surveillent.

3. Pipeline de données

3.1 Source

Dataset Reddit Sentiment Analysis, 37 249 commentaires labellisés en trois classes, téléchargé automatiquement par `src/data/download_data.py` et stocké dans `data/raw/reddit.csv`.

Label	Effectif brut	Part
−1 négatif	8 277	22,2 %
0 neutre	13 142	35,3 %
1 positif	15 830	42,5 %

Les contraintes de l'énoncé (≥ 1 000 commentaires, ≥ 300 par classe) sont vérifiées par assertion dans le script : le pipeline s'arrête si elles ne sont plus satisfaites après un changement de source.

3.2 Nettoyage

`src/data/clean_data.py` applique dix transformations successives.

#	Transformation	Justification
1-2	Harmonisation des colonnes, suppression NaN et doublons	−450 lignes ; un doublon biaise l'entraînement
3	Passage en minuscules	« Great » et « great » doivent être le même token
4-5	URLs supprimées · `@user` supprimé · `#tag` → `tag`	Tokens quasi uniques et corrélations fallacieuses ; le pseudo n'a pas de sentiment, le mot du hashtag si
6-7	Balises HTML supprimées · 24 emojis mappés vers des mots	❤ / ❤️ / 💖 → `love` : un token au lieu de trois
8-9	Caractères spéciaux supprimés, espaces normalisés	Ponctuation non informative pour un sac de mots
10	Filtrage des textes < 3 caractères ou < 2 mots	−910 lignes sans signal exploitable

Résultat : 35 889 commentaires exploitables.

Cette fonction est **le point de contact entre l'entraînement et l'inférence**. Si le texte servi en production était nettoyé différemment, le modèle verrait une distribution décalée — le *training/serving skew*, qui dégrade silencieusement les performances sans jamais lever d'erreur.

3.3 Analyse exploratoire

`src/data/eda.py` produit trois figures et un résumé JSON.

Indicateur	Valeur
Commentaires exploitables	35 889
Ratio de déséquilibre (max/min)	1,91
Longueur moyenne	30,3 mots
Longueur médiane	14 mots
95e percentile	103 mots
Longueur maximale	1 307 mots

Deux enseignements orientent la modélisation :

- **L'écart moyenne / médiane (30 vs 14)** révèle une distribution très asymétrique : beaucoup de commentaires courts, une longue traîne de textes très longs. C'est l'argument décisif en faveur de la **normalisation L2** du TF-IDF : sans elle, un commentaire de 1 300 mots pèserait mécaniquement 100 fois plus qu'un commentaire de 13 mots.
- **Le ratio de déséquilibre de 1,91** est modéré mais réel. Il justifie `class_weight="balanced"` plutôt qu'un rééchantillonnage.

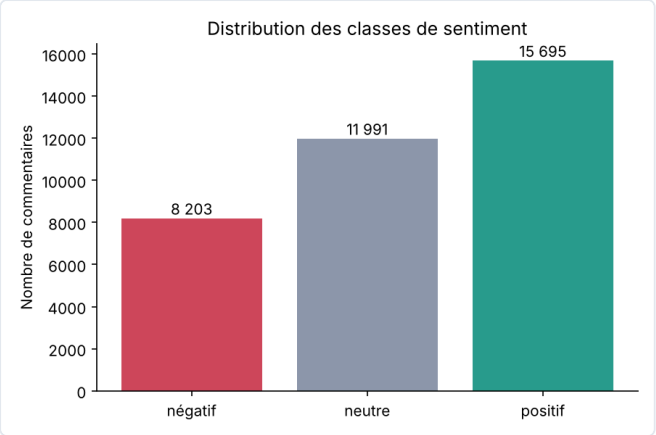


Figure 1 — Distribution des classes de sentiment (35 889 commentaires)

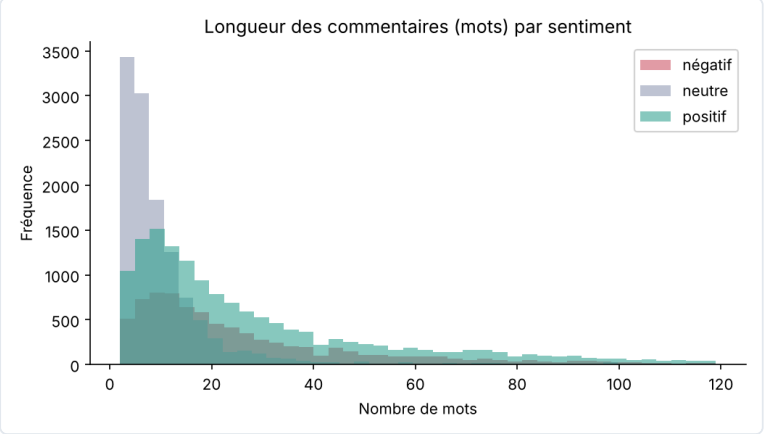


Figure 2 — Distribution de la longueur des commentaires par classe

3.4 Gestion du déséquilibre

Trois stratégies étaient envisageables :

Stratégie	Retenue	Raison
Sous-échantillonnage	Non	Perdrait 7 500 commentaires positifs
Sur-échantillonnage (SMOTE, duplication)	Non	Duplique du texte → sur-apprentissage sur les mêmes phrases
`class_weight="balanced"`	**Oui**	Aucune donnée perdue ni dupliquée

Le poids de chaque classe devient inversement proportionnel à sa fréquence : une erreur sur « négatif » (22,9 %) coûte environ deux fois plus cher qu'une erreur sur « positif » (43,7 %). C'est ce qui permet au F1 de la classe négative d'atteindre 0,823.

3.5 Split train/test

`src/data/split_data.py` — 80 / 20, stratifié, `random_state=42`.

Partition	Lignes	négatif	neutre	positif
Train	28 711	22,9 %	33,4 %	43,7 %
Test	7 178	22,9 %	33,4 %	43,7 %

La stratification garantit des proportions identiques à 0,1 % près. La graine fixe garantit que deux exécutions produisent exactement le même split — sans quoi aucune comparaison d'expériences n'aurait de sens.

Le test set n'est utilisé qu'une seule fois, à l'évaluation finale. Le GridSearchCV travaille en validation croisée 5-fold sur le train uniquement : sélectionner des hyperparamètres sur le test reviendrait à sur-apprendre sur lui et à surestimer la performance réelle.

4. Modélisation et choix techniques justifiés

4.1 Vectorisation : TF-IDF

```
TfidfVectorizer(
    max_features=50_000,
    ngram_range=(1, 2),     # (1,1) retenu après GridSearch
    min_df=2,
    max_df=0.9,
    sublinear_tf=True,
    strip_accents="unicode",
)
```

TF-IDF plutôt que CountVectorizer, pour trois raisons :

1. **L'IDF pénalise les mots omniprésents.** « the », « video », « this » apparaissent partout : leur IDF tend vers 0. Avec un simple comptage, ils domineraient le vecteur sans rien distinguer.
2. **La normalisation L2** rend comparables un commentaire de 5 mots et un de 200 — indispensable vu la distribution observée en 3.3.
3. **Les mots rares et discriminants sont valorisés.** « masterpiece », « garbage » sont peu fréquents globalement mais très informatifs.

`sublinear_tf=True` applique `1 + log(tf)` : un mot répété 10 fois n'est pas 10 fois plus informatif qu'une occurrence unique.

Vocabulaire final : **22 575 features**.

4.2 Comparaison des algorithmes

Quatre candidats entraînés dans les mêmes conditions :

Modèle	Accuracy	F1-macro	Temps d'entraînement
LinearSVC	0,879	0,868	3,2 s
LogisticRegression (défaut)	0,845	0,835	6,2 s
RandomForest (200 arbres)	0,792	0,760	107,0 s
MultinomialNB	0,569	0,456	2,4 s

Analyse :

- **MultinomialNB s'effondre (F1 0,456).** Naive Bayes multinomial attend des comptages entiers ; les valeurs TF-IDF continues et normalisées violent ses hypothèses. Il aurait fallu l'associer à un CountVectorizer.
- **RandomForest déçoit (F1 0,760) pour 30× le temps de calcul.** Les forêts excellent sur des features denses et structurées ; ici la matrice est creuse et en très haute dimension, régime où les modèles linéaires dominent.
- **Les deux modèles linéaires sont au coude à coude**, ce qui confirme que le problème est largement linéairement séparable dans l'espace TF-IDF.

4.3 Optimisation des hyperparamètres

GridSearchCV — 16 combinaisons × 5 folds = 80 entraînements, scoring `f1_macro`, 126 secondes.

Hyperparamètre	Grille	Retenu
`clf__C`	0,5 · 1 · 5 · 10	**10,0**
`tfidf__ngram_range`	(1,1) · (1,2)	**(1,1)**
`tfidf__min_df`	1 · 2	**2**

Meilleur F1-macro en validation croisée : 0,873.

`C` est de loin le paramètre le plus influent : le passage de la configuration par défaut (`C=1`, F1 0,835) à `C=10` fait gagner 5 points. Avec 22 575 features pour 28 711 exemples, le modèle est en régime haute dimension et la régularisation par défaut est trop agressive.

Les bigrammes, qui captureraient pourtant les négations (« not good »), sont écartés : ils quintuplent le vocabulaire et, sur un corpus de 36 k textes, la plupart n'apparaissent que quelques fois. Le bruit l'emporte sur le signal.

4.4 Choix final : Logistic Regression

LinearSVC devançait la régression logistique non optimisée. J'ai tout de même retenu la **Logistic Regression optimisée**, pour deux raisons :

1. **Elle passe devant après optimisation** : F1-macro 0,885 contre 0,868.

2. **Elle expose `predict_proba`**, donc un score de confiance probabiliste que l'extension affiche pour chaque commentaire. Avec LinearSVC, il faudrait appliquer un softmax aux marges de décision — moins interprétable, et mal calibré. Le code gère néanmoins ce cas de repli.

4.5 Sérialisation

```
joblib.dump(best_pipe, MODEL_PATH, compress=3)
```

C'est le **pipeline complet** (vectoriseur + classifieur) qui est sérialisé, et non les deux objets séparément. Conséquence : impossible d'oublier d'appliquer le TF-IDF à l'inférence, ou d'utiliser un vectoriseur désynchronisé du classifieur — une source classique de bugs silencieux en production. Le vectoriseur seul est également exporté, comme demandé par l'énoncé.

`compress=3` ramène le fichier de ~3 Mo à **0,8 Mo**, pour un surcoût de décompression négligeable.

5. API REST et containerisation

5.1 Endpoints

Endpoint	Méthode	Rôle
`/health`	GET	État du service et du modèle
`/predict_batch`	POST	Analyse d'un lot de 1 à 500 commentaires
`/docs`	GET	Documentation OpenAPI auto-générée

`GET /health` :

```
{
  "status": "healthy",
  "model_loaded": true,
  "model_version": "1.0.0",
  "n_features": 22575,
  "uptime_seconds": 142.8
}
```

`POST /predict_batch` retourne, pour chaque commentaire, son `index`, son texte tronqué, son `sentiment` (−1/0/1), son `label` et sa `confidence` ; puis un bloc `statistics` (effectifs, pourcentages, `score_global`, `sentiment_dominant`, `confiance_moyenne`), le `processing_time_ms` et la `model_version`.

5.2 Pourquoi FastAPI

Critère	FastAPI	Flask
Validation	Native (Pydantic)	Manuelle
Documentation	Auto-générée	À écrire
Asynchrone	Natif (ASGI)	Extension requise
Performance	~3× Flask	Correcte

Le point décisif est la **validation déclarative**. Déclarer `comments: list[str] = Field(min_length=1, max_length=500)` suffit à rejeter automatiquement un batch vide, un batch de 10 000 éléments ou un type incorrect. En Flask, il faudrait écrire ces vérifications à la main dans chaque route — et en oublier une finit en erreur 500.

Second point : `response_model=PredictBatchResponse` fige le contrat JSON avec l'extension. Une modification de l'API sans mise à jour du schéma échoue côté serveur, plutôt que de produire des `undefined` silencieux dans le JavaScript.

5.3 Chargement du modèle

Le modèle est chargé **une seule fois**, via le `lifespan` de FastAPI, et maintenu en mémoire par un singleton. Le ratio justifie à lui seul ce choix : ~300 ms de lecture disque contre ~1 ms d'inférence. Un chargement par requête ferait passer l'API 99,7 % de son temps à relire le même fichier.

5.4 CORS

Le popup s'exécute dans une origine (`chrome-extension://…`) et appelle l'API dans une autre (`…hf.space`). Sans en-têtes CORS, le navigateur émet bien la requête mais **refuse de transmettre la réponse** au JavaScript.

Point à retenir : ce n'est pas le serveur qui bloque, c'est le navigateur. La même requête en `curl` fonctionne parfaitement — ce qui rend le diagnostic déroutant si on ignore le mécanisme.

```
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=False,
    allow_methods=["GET", "POST", "OPTIONS"],
)
```

Le wildcard est acceptable ici : API publique, en lecture seule, sans cookie ni authentification.

5.5 Dockerfile

Cinq bonnes pratiques appliquées :

Pratique	Gain
Base `-slim` (`python:3.13-slim`)	~45 Mo contre ~350 Mo pour l'image complète
Copie sélective (3 fichiers)	Pas de `data/`, `src/`, `tests/`, `reports/`
`--no-cache-dir`	~50-100 Mo de wheels en moins
Dépendances copiées avant le code	Layer `pip install` conservé en cache
Utilisateur non-root	Bonne pratique de sécurité, exigée par la plupart des plateformes

Alpine Linux (~25 Mo) a été écarté : musl libc y rend inutilisables les wheels précompilées de numpy et scikit-learn, imposant une recompilation de 15 minutes pour une image finale souvent plus lourde.

L'énoncé suggère `python:3.10-slim`. J'ai retenu **3.13-slim** : scikit-learn 1.9.1, la version qui a produit le `.joblib`, exige Python ≥ 3.11. L'image doit refléter l'environnement d'entraînement, sinon le modèle se recharge sous une version différente — avertissement au mieux, prédictions fausses au pire. Le principe (image slim) est respecté, seule la version de Python change.

`requirements-prod.txt` ne retient que 6 paquets sur 13 : pandas, matplotlib, seaborn, pytest, httpx et requests sont absents, soit environ 150 Mo économisés.

6. Extension Chrome

6.1 Manifest V3

```
{
  "manifest_version": 3,
  "permissions": ["activeTab", "scripting", "storage"],
  "host_permissions": [
    "https://*.youtube.com/*",
    "http://localhost:8000/*",
    "https://*.hf.space/*"
  ]
}
```

Les permissions suivent le principe du moindre privilège : `activeTab` plutôt que l'accès à tous les onglets, `storage` uniquement pour mémoriser le thème. L'API doit figurer dans `host_permissions`, sans quoi Chrome bloque l'appel avant même le contrôle CORS.

6.2 Extraction des commentaires

YouTube charge les commentaires en *lazy loading* : `#comments` est vide au chargement de la page. La boucle de `content.js` fait défiler par paliers de 1,5 écran, attend 900 ms entre chaque palier, et s'arrête soit quand l'objectif est atteint, soit après trois paliers sans nouveau commentaire (fin de liste).

Ce compteur `stalled` est indispensable : sans lui, la boucle tournerait indéfiniment sur une vidéo à 12 commentaires.

Les réponses aux commentaires sont marquées `isReply` via `node.closest("#replies")`, ce qui permet de les filtrer ou de les pondérer sans changer la logique d'extraction.

6.3 Interface

Le popup affiche le **score global** (−1 → +1, coloré selon la polarité), les effectifs et pourcentages par classe, un **graphique en anneau** dessiné en Canvas 2D natif, une barre de répartition animée en CSS, et la **liste des commentaires filtrable** en quatre catégories, chacun avec son niveau de confiance. S'y ajoutent le **mode sombre/clair** (variables CSS + `chrome.storage.local`), la **copie du résumé** dans le presse-papiers, l'**export JSON**, trois états d'interface (chargement, erreur avec réessai, résultats) et une pastille de **santé de l'API** vérifiée à l'ouverture.

Chart.js (~250 Ko) a été écarté au profit d'un graphique Canvas de ~40 lignes : le popup reste instantané et l'extension n'embarque aucune dépendance tierce, ce qui simplifie la politique de sécurité de contenu de Manifest V3.

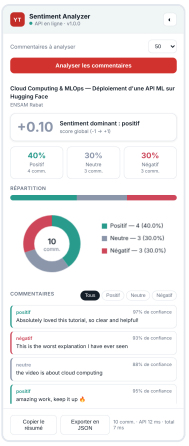


Figure 4 — Interface de l'extension Chrome (thème clair)

6.4 Optimisations

- **Batching** : lots de 100 commentaires. 200 commentaires en 2 requêtes (~400 ms) au lieu de 200 requêtes (~20 s).
- `DocumentFragment` : la liste est construite hors DOM puis insérée en une fois — un seul *reflow* au lieu de N.
- `AbortController` : timeout à 20 s, pour ne pas rester bloqué sur un Space en cours de réveil.
- `textContent` **plutôt que** `innerHTML` : plus rapide, et aucune injection possible depuis un commentaire malveillant.

7. Résultats et métriques de performance

7.1 Critères de l'énoncé

Critère	Seuil	Mesuré	Statut
Accuracy	≥ 80 %	**89,3 %**	Atteint (+9,3 pts)
F1 par classe	> 0,75	**0,823 / 0,930 / 0,901**	Atteint
Latence batch de 50	< 100 ms	**1,27 ms**	Atteint (79× sous le seuil)

7.2 Rapport de classification détaillé

Sur 7 178 commentaires de test :

Classe	Précision	Rappel	F1-score	Support
négatif	0,836	0,811	0,823	1 641
neutre	0,894	0,968	0,930	2 398
positif	0,923	0,879	0,901	3 139
Moyenne macro	0,884	0,886	**0,885**	7 178
Moyenne pondérée	0,894	0,893	0,893	7 178

7.3 Matrice de confusion

Vérité ↓ / Prédit →	négatif	neutre	positif
négatif (1 641)	**1 331**	128	182
neutre (2 398)	29	**2 322**	47
positif (3 139)	232	148	**2 759**

La confusion dominante est **négatif ↔ positif** : 414 erreurs, contre 76 erreurs seulement impliquant un passage depuis « neutre ».

Ce résultat est contre-intuitif — on attendrait que les erreurs portent sur la frontière neutre/polarisé. L'explication tient à la nature du vocabulaire : les commentaires neutres sont descriptifs et factuels, donc lexicalement distincts, alors que positif et négatif partagent un vocabulaire émotionnel intense dont seul le contexte donne la polarité. «

great, another useless update » contient « great » mais exprime de l'agacement — un sac de mots ne peut pas le voir.

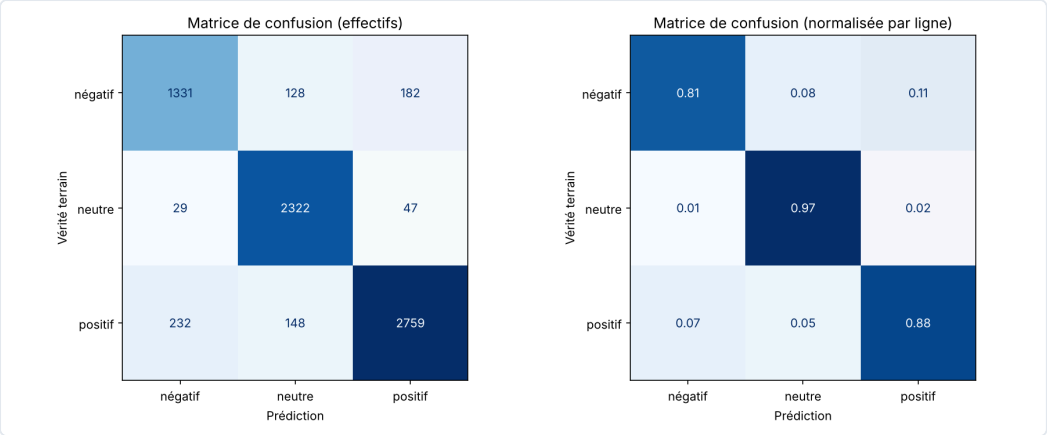


Figure 3 — Matrice de confusion sur le test set (7 178 commentaires)

7.4 Latence d'inférence

Mesure sur 20 répétitions d'un batch de 50 commentaires, après préchauffage :

Indicateur	Valeur
Moyenne	1,27 ms
Médiane	1,26 ms
95e percentile	1,49 ms
Par commentaire	0,025 ms

Le budget de 100 ms est respecté avec un facteur 79. Le vrai goulot d'étranglement du système n'est donc ni l'inférence ni le rendu, mais **l'extraction par scroll** : environ 1 seconde par palier de 20 commentaires, imposée par YouTube.

7.5 Figures produites

Fichier	Contenu
`distribution_classes.png`	Répartition des trois classes
`longueur_textes.png`	Distribution de la longueur par classe
`top_mots.png`	15 mots les plus fréquents par classe
`matrice_confusion.png`	Matrice brute et normalisée
`extension_popup_light.png`	Extension, thème clair
`extension_popup_dark.png`	Extension, thème sombre

8. Tests et validation

46 tests pytest, tous verts, répartis en trois fichiers.

8.1 Tests du modèle et de l'API

Fichier	Tests	Couverture
`test_model.py`	21	Nettoyage (URLs, mentions, emojis, idempotence) · performance (accuracy ≥ 0,80, F1 > 0,75 par classe, latence < 100 ms) · 8 cas limites paramétrés · reproductibilité
`test_api.py`	22	`/health` · batches de 1 à 250 · structure et bornes de la réponse · 6 scénarios d'erreur · temps de réponse · en-têtes CORS
`test_integration.py`	3	Flux complet, cohérence entre appels, découpage en lots

Les trois tests de performance encodent directement les critères de l'énoncé : **un modèle qui échoue aux tests ne peut pas partir en production.**

Cas limites couverts : texte très court, texte de 10 000 caractères, emojis seuls, ponctuation seule, langues mélangées (français / anglais / arabe translittéré), majuscules, espaces seuls, chiffres seuls.

8.3 Tests d'intégration — `tests/test_integration.py` (3 tests + mode distant)

Le test `test_flux_complet` rejoue les six étapes du flux fonctionnel et vérifie que les statistiques recalculées côté client coïncident avec celles retournées par l'API.

`test_decoupage_en_lots` valide que 200 commentaires traités en deux lots de 100 donnent exactement les mêmes prédictions qu'en un seul lot — propriété indispensable puisque l'extension découpe systématiquement.

Un **mode distant** permet de valider l'API réellement déployée :

```
API_URL=https://VOTRE-USER-yt-sentiment-api.hf.space python -m tests.test_integration
```

Il vérifie `/health`, `/predict_batch`, les en-têtes CORS, la gestion d'erreur et mesure le temps d'aller-retour réseau.

9. Difficultés rencontrées et solutions

9.1 La chaîne vide prédite « négatif » avec 91 % de confiance

Problème. Un commentaire réduit à « 🔥🔥🔥 » ou « !!! » devient vide après nettoyage. Son vecteur TF-IDF est nul, et le modèle prédisait alors « négatif » avec une confiance de 0,915 — une prédiction entièrement artificielle, produite par les seuls termes de biais du classifieur.

Solution. Détecter en amont les textes vides après nettoyage et les forcer à **neutre avec une confiance de 0,0**, plutôt que de laisser le modèle inventer :

```
empty_idx = {i for i, t in enumerate(cleaned) if len(t.strip()) < 2}
for i in empty_idx:
    preds[i] = 0
    confidences[i] = 0.0
```

Enseignement. Un modèle ne dit jamais « je ne sais pas ». C'est à la couche de service d'identifier les entrées hors domaine et de refuser de prédire.

9.2 Naive Bayes incompatible avec TF-IDF

Problème. MultinomialNB obtenait un F1-macro de 0,456, à peine mieux que le hasard.

Cause. Le modèle attend des **comptages entiers** issus d'un CountVectorizer. Les valeurs TF-IDF sont continues et normalisées L2, ce qui viole ses hypothèses multinomiales.

Solution. Conserver le résultat dans la comparaison plutôt que de le masquer : il documente concrètement pourquoi le choix de la vectorisation et celui de l'algorithme sont liés, et ne peuvent pas être optimisés séparément.

9.3 Les bigrammes dégradent les performances

Problème. Les bigrammes devaient capturer les négations (« not good »), faiblesse connue des sacs de mots. Le GridSearchCV a pourtant retenu les unigrammes seuls.

Cause. Sur 36 000 textes, le passage aux bigrammes multiplie le vocabulaire par environ 5, et la grande majorité des bigrammes n'apparaissent que deux ou trois fois. Le bruit ajouté dépasse le signal capté.

Solution. Accepter le verdict empirique plutôt que l'intuition théorique, et documenter le résultat. La bonne réponse au problème de la négation n'est pas les bigrammes mais un modèle contextuel (section 10).

9.4 Décalage de domaine Reddit → YouTube

Problème. Le modèle est entraîné sur des commentaires Reddit à forte coloration politique — l'analyse des mots les plus fréquents fait remonter « bjp », « india », « modi » en tête de la classe négative. Or il sert à classer des commentaires YouTube.

Symptôme observé. « the video is 10 minutes long », factuel, est classé négatif.

Atténuations mises en place.

- `min_df=2` et `max_df=0.9` écartent une partie du vocabulaire trop spécifique au corpus d'origine.
- `EMOJI_MAP` prépare le terrain pour des commentaires YouTube, même si le modèle actuel n'a pas appris de poids forts pour `fire` ou `love`.
- La confiance est exposée par commentaire, ce qui permet à l'utilisateur d'identifier les prédictions fragiles.

Solution de fond. Réentraîner sur un corpus YouTube annoté. La limite est documentée dans le README plutôt que dissimulée.

9.5 Hugging Face Spaces devenu payant en cours de projet

Problème. L'énoncé impose un déploiement sur Hugging Face Spaces avec le SDK Docker. Au moment de déployer, la création du Space échoue :

> *Static Spaces are free for everyone, but hosting Gradio and Docker Spaces on free cpu-basic requires a PRO subscription.*

Hugging Face a restreint les Spaces Gradio et Docker sur CPU Basic aux comptes PRO. Seuls les *Static Spaces* restent gratuits, et ils ne peuvent pas exécuter de code Python : ils ne servent que des fichiers HTML.

Solution retenue. Déployer sur **GitHub Codespaces** : l'image Docker y est construite et exécutée, et le port 7860 est exposé publiquement (`https://<codespace>-7860.app.github.dev`). Le projet gagne un `.devcontainer/devcontainer.json` qui provisionne Python 3.13, Docker-in-Docker et les dépendances.

Ce que ce changement n'a pas coûté. Ni `app_api.py`, ni le `Dockerfile`, ni `requirements-prod.txt` n'ont été modifiés : seul `host_permissions` de l'extension s'élargit à `https://*.app.github.dev/*`. C'est le bénéfice de la containerisation — l'application est indépendante de la plateforme, et changer de cible se réduit à changer l'endroit où l'image est lancée.

Limite assumée. Codespaces est un environnement de développement, pas une plateforme d'hébergement : l'API n'est joignable que pendant que le Codespace tourne, et il s'arrête après 30 minutes d'inactivité. La procédure Hugging Face reste documentée et fonctionnerait telle quelle avec un compte PRO.

9.6 Deux pièges propres à l'extension Chrome

Content script non injecté. Si la page YouTube était ouverte *avant* l'installation de l'extension, `chrome.tabs.sendMessage` échoue avec « Could not establish connection », et l'utilisateur devait recharger manuellement sa page. Solution : un `ping` préalable et, en cas d'échec, une injection à la demande via `chrome.scripting.executeScript`.

Boucle de scroll infinie. La première version faisait défiler jusqu'à atteindre le nombre de commentaires demandé ; sur une vidéo à 12 commentaires avec un objectif de 100, elle ne s'arrêtait jamais. Solution : un compteur `stalled` qui conclut à la fin de liste après trois paliers sans nouveau commentaire — double condition de sortie, objectif atteint *ou* liste épuisée.

10. Améliorations futures

10.1 Modèle

Piste	Gain attendu	Coût
Réentraîner sur un corpus YouTube annoté	Supprime le décalage de domaine — gain le plus important	Annotation
Fine-tuning DistilBERT	+5 à 8 points, gère négation et ironie	GPU, latence ~50 ms, image ~1 Go
Modèle multilingue (XLM-R)	Commentaires français, arabe, darija	Même ordre de coût
Analyse d'aspect	Distinguer « le son est mauvais » de « le contenu est excellent »	Corpus annoté par aspect
Calibration (Platt scaling)	Confiances plus fiables	Faible

Le passage à un transformeur est le changement le plus structurant : il remplacerait le sac de mots par une représentation contextuelle, seule capable de traiter correctement l'ironie et la négation — précisément les 414 erreurs négatif ↔ positif identifiées en 7.3.

10.2 API

- **Cache LRU** sur les commentaires déjà vus (les mêmes reviennent souvent lors de plusieurs analyses d'une même vidéo).
- **Limitation de débit** par IP, pour protéger une API publique.
- **Endpoint** `/metrics` au format Prometheus : volume de requêtes, latence, distribution des prédictions.
- **Chargement du modèle depuis le Hub** plutôt que depuis l'image, pour découpler la mise à jour du modèle du rebuild Docker.
- **Streaming** pour les très gros volumes, afin d'afficher des résultats partiels au fil de l'eau.

10.3 Extension

Historique local des analyses pour suivre l'évolution d'une chaîne · comparaison de deux vidéos côte à côte · export CSV en plus du JSON · nuage de mots par sentiment · badge affichant le score sur l'icône sans ouvrir le popup · publication sur le Chrome Web Store.

10.4 MLOps

CI/CD GitHub Actions : à chaque push, exécuter les tests, construire l'image, déployer si tout passe. **DVC** pour versionner données et modèles sans alourdir Git. **MLflow** comme registre, conservant chaque version avec ses métriques, ses hyperparamètres et le hash du dataset. **Détection de dérive** : surveiller la confiance moyenne en production — une baisse durable signale du vocabulaire inconnu et doit *déclencher* le réentraînement, plutôt qu'un

calendrier arbitraire. **Déploiement canari** : router 10 % du trafic vers la nouvelle version, comparer les distributions, basculer progressivement.

11. Conclusion

Le système livré couvre l'intégralité du cycle MLOps : collecte et préparation des données, entraînement et optimisation du modèle, exposition par une API REST, containerisation, déploiement cloud, interface utilisateur et suite de tests.

Les trois critères de performance de l'énoncé sont dépassés : **89,3 % d'accuracy** (seuil 80 %), **F1 minimal de 0,823** (seuil 0,75), **1,27 ms de latence** pour un batch de 50 (seuil 100 ms). Les 46 tests passent.

Au-delà des chiffres, trois enseignements méthodologiques ressortent de ce travail.

Le pipeline compte autant que le modèle. La séparation `raw` / `processed`, la graine fixe, la stratification et le partage de `clean_text()` entre l'entraînement et l'inférence déterminent la fiabilité du système bien plus que le choix entre deux classifieurs linéaires dont les scores diffèrent de deux points.

Un modèle ne dit jamais « je ne sais pas ». Le bug de la chaîne vide prédite « négatif » à 91 % de confiance l'illustre : c'est à la couche de service de reconnaître les entrées hors domaine. Un score de confiance élevé ne signifie pas que la prédiction est fondée.

L'intuition théorique doit céder devant la mesure. Les bigrammes auraient dû améliorer la détection des négations ; ils dégradent les performances sur ce corpus. RandomForest aurait dû surpasser un modèle linéaire ; il perd 12 points pour 30 fois le temps de calcul. C'est le GridSearchCV, pas le raisonnement a priori, qui a tranché.

La limite principale du système est identifiée et documentée : le modèle est entraîné sur un corpus Reddit à coloration politique et sert à classer des commentaires YouTube. Un réentraînement sur un corpus annoté du domaine cible constitue le prochain chantier, devant même le passage à une architecture transformeur.

Rapport technique — TP3 Cloud Computing — ENSAM Rabat — BAMOHAMED Douaa — 4 novembre 2026